التشغيل على الـ Mac
الفصل 2 · Terminal

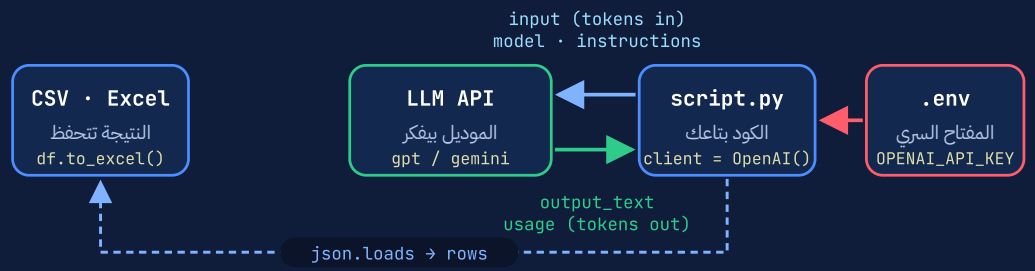
- أنهي Python ف2 — `python3 --version`
- اعمل بيئة للمشروع ف2 ★ — `python3 -m venv .venv`
- فعّلها (deactivate تقفلها) ف2 ★ — `source .venv/bin/activate`
- سطّب جوه الـ venv ف2 ★ — `python -m pip install openai`
- ثبّت النسخ ف2 — `pip freeze > requirements.txt`
- سطّب من الملف ف2 — `pip install -r requirements.txt`
- اقرا الخيارات ف13 — `python script.py --help`
- مفتاح للجلسة دي بس ف11 ⚠ — `export OPENAI_API_KEY=sk-...`

Python ↔ JavaScript
ف3 · ف4 · Python vs JS

المعنى	Python	JavaScript
قايمة	`list [1, 2]`	`Array [1, 2]`
مفتاح وقيمة	`dict {"a": 1}`	`Object {a: 1}`
قيم خاصة	`None · True`	`null · true`
دالة	`def f(x):`	`function f(x) {}`
نص فيه متغير	`f"{n} files"`	`` `${n} files` ``
الطول	`len(xs)`	`xs.length`
لوب	`for x in xs:`	`for (const x of xs)`
الأخطاء	`try: ... except:`	`try {} catch {}`
تعليق	`# comment`	`// comment`
منطق	`and · or · not`	`&& · || · !`

الـ Indentation في Python هو الأقواس: 4 مسافات = جوه البلوك.

اسكريبت AI بيشتغل إزاي
ف11 · ف14 · ف16 · Script → LLM → File



بتدفع على الـ Tokens الداخلة والخارجة. والمفتاح يفضل في .env، عمره ما يتكتب في الكود.

أساسيات اللغة
ف4 – ف6 · Python basics

```python
name = "Sara"; n = 3
items = [x * 2 for x in range(n)]
user = {"name": name, "age": 30}
age = user.get("age", 0)
if age >= 18 and name:
    print(f"{name}: {age:,.1f}")
for i, x in enumerate(items, 1):
    print(i, x)
def cost(tokens: int, *, rate=0.5) -> float:
    return tokens / 1_000_000 * rate
```

- Slice: أول 100 · آخر 20 ف4 — `text[:100] · text[-20:]`
- import حاجة من مكتبة ف7 ★ — `from pathlib import Path`
- شغّل لو الملف اتشغّل مباشرة ف2 — `if __name__ == "__main__":`

الملفات والبيانات
ف9 · ف10 · Files & data

- مسار (بيشتغل على أي جهاز) ف9 ★ — `Path("data") / "a.md"`
- اقرا ملف نص ف9 ★ — `p.read_text(encoding="utf-8")`
- اكتب (فوق القديم!) ف9 ⚠ — `p.write_text(s, encoding="utf-8")`
- كل الملفات بترتيب ف9 — `sorted(folder.glob("*.md"))`
- أجزاء الاسم ف9 — `p.stem · p.suffix · p.name`
- JSON من وإلى نص ف10 ★ — `json.loads(s) · json.dumps(o, ensure_ascii=False)`
- اقرا Excel ف10 — `pd.read_excel(p)`
- اكتب Excel ف10 — `df.to_excel(p, index=False)`
- CSV يفتح عربي في Excel ف10 — `df.to_csv(p, encoding="utf-8-sig")`

أي API بـ requests و env.
الفصل 11

- حمّل env. للبيئة ف11 ★ — `load_dotenv()`
- طلب GET ف11 — `requests.get(url, params=q, timeout=30)`
- اتأكد ونسّق ف11 ★ — `r.raise_for_status() · r.json()`
- المفتاح عمره ما يترفع ف11 ⚠ — `echo .env >> .gitignore`

OpenAI SDK
الفصل 16 · Responses API

```python
from openai import OpenAI
client = OpenAI(max_retries=4, timeout=60)
r = client.responses.create(
    model="gpt-6-luna",
    instructions="Answer briefly",
    input=text,
    max_output_tokens=500,
)
print(r.output_text, r.usage)
```

- تفكير أقل = أرخص وأسرع ف16 — `reasoning={"effort": "low"}`
- Structured بـ Pydantic ف21 ★ — `client.responses.parse(..., text_format=Model)`
- الـ Object جاهز ف21 — `r.output_parsed`
- RAG ل Embeddings ف24 — `client.embeddings.create(model=, input=[...])`

تشريح سطر: client.responses.create
ف8

```
client . responses . create ( model=
                  input= ... )
```

1. الـ Object اللي فيه المفتاح
2. الـ API (القسم)
3. الـ Method: ابعت طلب
4. أنهي موديل
5. كلام المستخدم
6. وباقي الإعدادات

الأخطاء
الفصل 12 · try / except

```python
try:
    r = client.responses.create(...)
except openai.RateLimitError as e:
    time.sleep(10)        # retry later
except Exception as e:
    log.error("failed %s: %s", path, e)
    raise
finally:
    save_progress()
```

Gemini SDK `google-genai` — الفصل 17

```python
from google import genai
from google.genai import types
client = genai.Client()   # GEMINI_API_KEY
r = client.models.generate_content(
    model="gemini-3.8-flash",
    contents=text,
    config=types.GenerateContentConfig(
        system_instruction="Answer briefly",
        max_output_tokens=500),
)
print(r.text, r.usage_metadata)
```

- ف21 JSON (مع response_json_schema): `response_mime_type="application/json"`
- ف23 ابعت صورة أو PDF: `types.Part.from_bytes(data=, mime_type=)`
- ف27 عدّ قبل ما تبعت: `client.models.count_tokens(model=, contents=)`

OpenRouter واحد API — الفصل 19

- ★ ف19 نفس SDK بتاع OpenAI: `OpenAI(base_url="https://openrouter.ai/api/v1")`
- ف19 الشركة/الموديل: `model="openai/gpt-6-luna"`
- ف19 Fallback لو واحد وقع: `extra_body={"models": [a, b]}`

Structured output مضمون JSON — الفصل 21

```python
from pydantic import BaseModel
class Item(BaseModel):
    title: str
    tags: list[str]
r = client.responses.parse(
    model="gpt-6-luna", input=text,
    text_format=Item)
item = r.output_parsed   # Item object
```

على نطاق واسع Scale & safety — ف25 – ف27

- ★ ف26 جرّب على شوية الأول: `--dry-run · --limit 5`
- ف26 توازي بسيط: `ThreadPoolExecutor(4)`
- ف26 async بحد أقصى: `asyncio.Semaphore(8) + gather`
- ★ ف26 JSONL: كمّل من مكان ما وقف: `open(p, "a") + f.flush()`
- ف25 Backoff لـ Retry: `min(cap, base * 2**n) + jitter`
- ★ ف27 التكلفة (السعر لكل مليون): `tokens / 1_000_000 * price`
- ف26 نص السعر لو مش مستعجل: `Batch API`

Function calling Tools — الفصل 22

- ف22 عرّف الدوال للموديل: `tools=[{"type": "function", ...}]`
- ف22 الموديل طلب إيه: `call.name · json.loads(call.arguments)`
- ف22 رجّع النتيجة ليه: `function_call_output`

logging g argparse CLI — الفصل 13

- ف13 Option برقم: `p.add_argument("--limit", type=int)`
- ف13 Flag من غير قيمة: `p.add_argument("--dry-run", action="store_true")`
- ف13 لوج بدل print: `logging.basicConfig(level="INFO")`

الـ Prompts في الكود Templates — الفصل 20

- ف20 قالب فيه فراغات: `PROMPT.format(text=t)`
- ف20 الـ Prompt في ملف بنسخة: `prompts/v2.md`
- ★ ف20 أمثلة قبل الطلب: `few-shot: 2-3 examples`

حاجات بتتلخبط بينها X vs Y

قارن بسرعة

بدل	الفرق	خلي بالك	ف
اللغة			
`=` / `==`	تعيين قيمة / مقارنة	غلط if x = 1	ف4
`list` / `tuple` / `dict`	تتغير / ثابتة / مفتاح وقيمة		ف4
`print` / `return`	يطبع على الشاشة / يرجّع قيمة للكود	الدالة اللي مبترجعش = None	ف6
`json.load` / `json.loads`	من ملف / من نص (s = string)		ف10
`"w"` / `"a"`	يكتب فوق / يضيف في الآخر	للـ Resume استخدم a	ف9
البيئة			
`pip` / `python -m pip`	أي pip / pip بتاع الـ Python ده بالظبط	التاني أضمن	ف2
Homebrew Python / venv	بتاعة النظام / بيئة لكل مشروع	متسطبش برة venv	ف2
الـ AI			
instructions / input	قواعد ثابتة (system) / كلام المستخدم	الـ Prompt الثابت في instructions	ف14
Responses / Chat Completions	الـ API الجديد / القديم	الجديد للمشاريع الجديدة	ف16
input / output tokens	اللي بتبعته / اللي الموديل بيكتبه	الـ output أغلى	ف27
Threads / asyncio / Batch	بسيط / كتير جدًا / أرخص بس أبطأ	ابدأ بـ Threads	ف26

لو حصل كذا... اعمل كذا Troubleshooting — الفصل 12

اقرا الـ Traceback من **تحت لفوق**: آخر سطر هو الخطأ، واللي فوقه على طول هو السطر بتاعك.

- ف2 **externally-managed-environment** · بتسطّب برة venv
  - `python3 -m venv .venv`
  - `source .venv/bin/activate`
- ف2 **ModuleNotFoundError** · الـ Python ده مش شايف المكتبة
  - `which python`
  - `python -m pip install x`
- ف1 **IndentationError** · مسافات زيادة أو ناقصة
  - `4 spaces · no tabs`
- ف7 **ImportError من ملف عندك** · سمّيت ملفك openai.py
  - `mv openai.py my_test.py`
- ف9 **UnicodeDecodeError** · الملف مش utf-8
  - `open(p, encoding="utf-16")`
- ف10 **JSONDecodeError** · الموديل رد بكلام مش JSON
  - `text_format=Model`
- ف20 **KeyError مع format.** · أقواس JSON جوه القالب
  - `{{ "title": ... }}`
- ف25 **401 · 429** · المفتاح غلط · كتير ورا بعض
  - `load_dotenv()`
  - `max_retries=4`